

CipherSchools

Website: <https://cipherschools.com> | Email: support@cipherschools.com | Contact: +91-7206810246

Assignment Brief — AI Code Review Agent

Build an autonomous agent that clones a repository, analyzes code with AST parsing, and generates confidence-rated review comments.

Duration: 3 Days | Difficulty: Advanced | Domain: Agentic AI | Type: Project + Demo

About the Assignment:

This assignment challenges you to build a fully functional AI-powered code review agent from scratch in 3 days. The agent must autonomously clone a GitHub repository, parse source files using Abstract Syntax Trees (AST), submit code to a Large Language Model (LLM) for analysis, and produce structured review comments — complete with severity ratings and confidence scores.

This is not a tutorial project. You are expected to make real engineering decisions: how to chunk large files, how to design prompts that yield consistent structured output, how to handle edge cases gracefully, and how to present results in a compelling, usable interface.

Objective:

Build an end-to-end agentic AI pipeline that reviews code repositories and outputs actionable, confidence-rated review comments via a dashboard UI.

Deliverable:

A deployed Streamlit application with a live demo URL, a GitHub repository with clean commits, and a 2-minute screen-recording walkthrough.

Creative Twist:

Every comment the agent generates must include a self-rated confidence score (0–100%). Low-confidence comments are shown separately with a 'verify this' label — demonstrating production-grade epistemic humility.

Learning Objectives:

Upon successful completion of this assignment, you will demonstrate:

1. Agentic AI: Proficiency in designing and orchestrating a multi-step LLM pipeline (ingestion → parsing → review → output).

2. Code Analysis: Working knowledge of Abstract Syntax Tree (AST) parsing for at least one programming language (Python, JavaScript, or Go).
3. Prompt Engineering: Ability to engineer prompts that produce consistent, schema-valid JSON output from an LLM.
4. API Integration: Experience integrating external APIs (GitHub API or PyGitHub) into a functional workflow.
5. UI / Deployment: Capability to build and deploy a user-facing dashboard with real-time feedback.
6. Responsible AI: Understanding of responsible AI practices — including uncertainty quantification via confidence scoring.

Required Tech Stack:

The following technologies are required. You may add additional libraries but may not replace the core stack without prior approval.

1. Ingestion: GitPython — Clone and validate GitHub repositories.
2. Parsing: Python ast / tree-sitter — Extract functions, classes, imports via AST.
3. LLM Review: OpenAI GPT-4o-mini or Claude Sonnet — Generate structured review comments.
4. Orchestration: Python (custom pipeline) — Coordinate ingestion → parse → review loop.
5. Output: Markdown + GitHub API — Format and optionally post PR comments.
6. Dashboard: Streamlit — Interactive UI with filters and download.
7. Deployment: Streamlit Cloud or HuggingFace Spaces — Live, shareable demo URL.

Submission Requirements:

1. GitHub repository URL (must be public or shared with the evaluator). Commit history must show meaningful, incremental commits across 3 days — not one giant final commit.
2. Live deployment URL (Streamlit Cloud, HuggingFace Spaces, or equivalent). The URL must be accessible without login.
3. A README.md covering: project overview, setup instructions, architecture diagram, known limitations, and what you would build next with more time.
4. 2-minute screen-recording demo (Loom, YouTube unlisted, or equivalent). Must show the agent running live, not a pre-recorded result.
5. (If claiming the GitHub API bonus) A screenshot or screen-recording showing inline PR comments posted to a real pull request.

Academic Integrity & AI Use Policy:

This assignment is about building AI systems, not just using them. The following rules apply:

1. You may use AI assistants (ChatGPT, Claude, Copilot, etc.) to help write individual code snippets. This is expected and encouraged.
2. You may not use AI to generate the entire project. The architecture decisions, prompt design, and integration logic must be yours.

3. All code in your repository must be code you understand and can explain. Evaluators may ask you to walk through any section during a viva/demo.
4. You must not copy another student's pipeline design or prompt templates. Structural similarity will be investigated.
5. Data sources and any third-party repositories used for testing must be cited in your README.

Grading Rubric:

Total: 100 points. The rubric rewards correctness, code quality, creative execution, and presentation — not just getting it to run.

1. Pipeline correctness (25 pts) — No crashes on 3 test repos; handles edge cases gracefully.
2. LLM prompt quality (20 pts) — Comments are meaningful, not hallucinated; JSON parses cleanly.
3. Confidence scoring (15 pts) — Bucketed display, visual indicator, 'verify' label for low-confidence.
4. Dashboard UI (15 pts) — Clean Streamlit UI; filter by category works; download works.
5. Code quality (10 pts) — Logical file organization; env vars for API keys; clear README.
6. Deployment (10 pts) — Streamlit Cloud or equivalent; tested externally.

Good luck. Build something you are proud to demo. The best projects are the ones where the agent finds a real bug in a real repo.

Confidential — For educational use only